

งานทบทวนก่อนเรียน

เรื่อง Array, Stack, Queue ด้วยภาษา Java

รายวิชา Design and Analysis of Algorithms

จัดทำโดย ชคริต แสนองค์การ

รหัสนักศึกษา 68122250070

1. วัตถุประสงค์ของงาน

- ทบทวนพื้นฐานการเขียนโปรแกรมภาษา Java ผ่านโครงสร้างข้อมูล Array, Stack และ Queue
- เลือกใช้โครงสร้างข้อมูลให้เหมาะสมกับปัญหา
- เขียนโปรแกรม Java เพื่อแก้ปัญหาจากสถานการณ์จริง
- อธิบายลำดับการทำงานของข้อมูล
- วิเคราะห์ Time Complexity เบื้องต้น
- เตรียมโปรเจกต์สำหรับส่งผ่าน GitHub Repository

2. กรณีศึกษาที่ 1: Array

โจทย์ให้คะแนนนักศึกษา 10 คนเป็น {6, 8, 4, 9, 7, 5, 10, 3, 8, 2} และให้หาคะแนนรวม ค่าเฉลี่ย คะแนนสูงสุด คะแนนต่ำสุด จำนวนคนที่ได้ตั้งแต่ 7 คะแนนขึ้นไป และหาผู้ที่ได้ต่ำกว่า 5 คะแนน

2.1 วิธีคิด

- วนดูคะแนนทุกตัวใน Array แล้วบวกเข้าตัวแปร sum เพื่อหาคะแนนรวม
- เริ่ม max และ min จากคะแนนตัวแรก แล้วเปรียบเทียบกับคะแนนตัวถัดไปที่ไล่ตัว
- ถ้าคะแนนมากกว่าหรือเท่ากับ 7 ให้เพิ่ม countPass
- หลังจากวนครบทุกคน ให้หา $average = sum / \text{จำนวนข้อมูล}$
- วนอีกรอบเพื่อดูว่าคะแนนใดต่ำกว่า 5 แล้วแสดง Student ตามลำดับ

2.2 คำนวณผลลัพธ์

รายการ	ผลลัพธ์
คะแนนรวม	62 คะแนน
คะแนนเฉลี่ย	6.2 คะแนน
คะแนนสูงสุด	10 คะแนน

รายการ	ผลลัพธ์
คะแนนต่ำสุด	2 คะแนน
ได้ตั้งแต่ 7 คะแนนขึ้นไป	5 คน
ต่ำกว่า 5 คะแนน	Student 3 (4), Student 8 (3), Student 10 (2)

2.3 โค้ด Java: ArrayCaseStudy.java

```
import java.util.Arrays;

public class ArrayCaseStudy {
    public static void main(String[] args) {
        int[] scores = {6, 8, 4, 9, 7, 5, 10, 3, 8, 2};

        int sum = 0;
        int max = scores[0];
        int min = scores[0];
        int countPass = 0;

        for (int score : scores) {
            sum += score;

            if (score > max) {
                max = score;
            }

            if (score < min) {
                min = score;
            }

            if (score >= 7) {
                countPass++;
            }
        }

        double average = (double) sum / scores.length;

        System.out.println("คะแนนของนักศึกษา: " + Arrays.toString(scores));
        System.out.println("คะแนนรวม = " + sum);
        System.out.println("คะแนนเฉลี่ย = " + average);
        System.out.println("คะแนนสูงสุด = " + max);
        System.out.println("คะแนนต่ำสุด = " + min);
        System.out.println("จำนวนนักศึกษาที่ได้ตั้งแต่ 7 คะแนนขึ้นไป = " + countPass);

        System.out.println("นักศึกษาที่ควรทบทวนเพิ่มเติม:");
        for (int i = 0; i < scores.length; i++) {
            if (scores[i] < 5) {
                System.out.println("Student " + (i + 1) + " ได้ " + scores[i] + " คะแนน");
            }
        }
    }
}
```

```

    }
  }
}

```

2.4 Time Complexity ของ Array

การหาผลรวม ค่าสูงสุด ค่าต่ำสุด และจำนวนคนที่ผ่าน ใช้การวนลูปผ่านคะแนนทั้งหมด 1 รอบ จึงเป็น $O(n)$ เมื่อ n คือจำนวนคะแนน ส่วนการหาผู้ที่ได้ต่ำกว่า 5 คะแนนมีอีก 1 รอบ จึงยังเป็น $O(n)$ เช่นเดิม เพราะค่าคงที่ไม่ทำให้ระดับความซับซ้อนเปลี่ยน

2.5 ตัวอย่างผลลัพธ์

คะแนนของนักศึกษา: [6, 8, 4, 9, 7, 5, 10, 3, 8, 2]

คะแนนรวม = 62

คะแนนเฉลี่ย = 6.2

คะแนนสูงสุด = 10

คะแนนต่ำสุด = 2

จำนวนนักศึกษาที่ได้ตั้งแต่ 7 คะแนนขึ้นไป = 5

นักศึกษาที่ควรทบทวนเพิ่มเติม:

Student 3 ได้ 4 คะแนน

Student 8 ได้ 3 คะแนน

Student 10 ได้ 2 คะแนน

3. กรณีศึกษาที่ 2: Stack

โจทย์เป็นระบบ Undo ในโปรแกรมจดบันทึก โดยคำสั่งที่ทำได้คือ Type Data, Type Structure, Delete Structure, Type Algorithm และ Type Java และต้อง Undo 2 ครั้ง

3.1 วิธีทำงาน

- สร้าง Stack<String> เพื่อเก็บคำสั่ง
- ใช้ push() เพิ่มคำสั่งตามลำดับที่ผู้ใช้ทำ
- แสดงคำสั่งทั้งหมดใน Stack
- ก่อน pop() ต้องตรวจสอบด้วย isEmpty() ก่อน เพื่อป้องกันการ pop จาก Stack ที่ว่าง
- Undo 2 ครั้งด้วย pop() ซึ่งจะนำคำสั่งล่าสุดออกก่อน
- แสดงคำสั่งที่ถูก Undo และแสดงสถานะของ Stack ที่เหลือ

3.2 ลำดับของ Stack

เมื่อเพิ่มคำสั่งครบ 5 คำสั่ง คำสั่งที่อยู่ด้านบนสุดคือ Type Java ดังนั้นเมื่อ Undo ครั้งที่ 1 จะเอา Type Java ออก และ Undo ครั้งที่ 2 จะเอา Type Algorithm ออก เหลือ 3 คำสั่ง คือ Type Data, Type Structure และ Delete Structure

3.3 โค้ด Java: StackCaseStudy.java

```
import java.util.Stack;

public class StackCaseStudy {
    public static void main(String[] args) {
        Stack<String> actions = new Stack<>();

        actions.push("Type Data");
        actions.push("Type Structure");
        actions.push("Delete Structure");
        actions.push("Type Algorithm");
        actions.push("Type Java");

        System.out.println("คำสั่งทั้งหมดใน Stack:");
        System.out.println(actions);

        System.out.println("\nทำ Undo 2 ครั้ง:");
        for (int i = 0; i < 2; i++) {
            if (!actions.isEmpty()) {
                System.out.println("Undo: " + actions.pop());
            } else {
                System.out.println("Stack ว่าง ไม่สามารถ Undo ได้");
            }
        }

        System.out.println("\nสถานะของ Stack หลังจาก Undo:");
        System.out.println(actions);

        System.out.println("หลักการที่ใช้: LIFO (เข้าทีหลัง ออกก่อน)");
    }
}
```

3.4 Time Complexity ของ Stack

การ push() และ pop() ทำที่ตำแหน่งบนสุดของ Stack จึงใช้ $O(1)$ ต่อ 1 ครั้ง ส่วน isEmpty() ก็เป็น $O(1)$

สำหรับการตรวจสอบสถานะของ Stack

3.5 ตัวอย่างผลลัพธ์

คำสั่งทั้งหมดใน Stack:

[Type Data, Type Structure, Delete Structure, Type Algorithm, Type Java]

ทำ Undo 2 ครั้ง:

Undo: Type Java

Undo: Type Algorithm

สถานะของ Stack หลังจาก Undo:

[Type Data, Type Structure, Delete Structure]

หลักการที่ใช้: LIFO (เข้าทีหลัง ออกก่อน)

3.6 การอธิบายหลักการ LIFO

Stack ใช้หลัก LIFO หรือ Last In, First Out หมายถึงข้อมูลทีเ่ใส่เข้ามาทีหลังจะถูกนำออกก่อน ตัวอย่างจากโจทย์คือ Type Java ใส่เข้ามาเป็นคำสั่งสุดท้าย จึงถูก Undo ก่อน Type Algorithm

4. กรณีศึกษาที่ 3: Queue

โจทย์เป็นระบบคิวผู้ป่วยในคลินิก มีผู้ป่วย P001 ถึง P005 จากนั้น P001 และ P002 ได้รับการแล้ว มี P006 และ P007 มาลงทะเบียนเพิ่ม และต้องการทราบผู้ป่วยคนถัดไปกับจำนวนผู้ป่วยที่ยังรอ

4.1 วิธีทำงาน

- สร้าง Queue<String> สำหรับเก็บหมายเลขผู้ป่วย
- เพิ่ม P001 ถึง P005 ตามลำดับ
- ตรวจสอบก่อน remove() ว่า Queue ไม่ว่าง แล้วเอา P001 และ P002 ออก
- เพิ่ม P006 และ P007 ต่อท้ายคิว
- ใช้ peek() ดูผู้ป่วยคนถัดไปโดยไม่เอาออกจาก Queue
- ใช้ size() เพื่อนับจำนวนผู้ป่วยที่ยังรออยู่
- แสดงสถานะ Queue หลังจากทำรายการทั้งหมด

4.2 สถานะของ Queue ตามลำดับ

เริ่มต้น: [P001, P002, P003, P004, P005] → remove P001 → [P002, P003, P004, P005] → remove P002
→ [P003, P004, P005] → add P006 และ P007 → [P003, P004, P005, P006, P007] ดังนั้นคนถัดไปคือ
P003 และมีผู้ป่วยรอ 5 คน

4.3 โค้ด Java: QueueCaseStudy.java

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueCaseStudy {
    public static void main(String[] args) {
        Queue<String> patients = new LinkedList<>();

        patients.add("P001");
        patients.add("P002");
        patients.add("P003");
        patients.add("P004");
        patients.add("P005");

        if (!patients.isEmpty()) {
            System.out.println("เรียกผู้ป่วย: " + patients.remove());
        }

        if (!patients.isEmpty()) {
            System.out.println("เรียกผู้ป่วย: " + patients.remove());
        }

        patients.add("P006");
        patients.add("P007");

        System.out.println("ผู้ป่วยคนถัดไป = " + patients.peek());
        System.out.println("จำนวนผู้ป่วยที่ยังรอ = " + patients.size());
        System.out.println("สถานะของ Queue = " + patients);

        System.out.println("หลักการที่ใช้: FIFO (มาก่อน ออกก่อน)");
    }
}
```

4.4 Time Complexity ของ Queue

สำหรับ Queue ที่เลือกใช้โครงสร้างที่เหมาะสม การ add(), remove(), peek() และ isEmpty() เป็น $O(1)$
ต่อการทำงาน 1 ครั้ง ส่วน size() ในตัวอย่างนี้ก็ใช้งานได้แบบ $O(1)$ กับ java.util.LinkedList

4.5 ตัวอย่างผลลัพธ์

เรียกผู้ป่วย: P001

เรียกผู้ป่วย: P002

ผู้ป่วยคนถัดไป = P003

จำนวนผู้ป่วยที่ยังรอ = 5

สถานะของ Queue = [P003, P004, P005, P006, P007]

หลักการที่ใช้: FIFO (มาก่อน ออกก่อน)

4.6 การอธิบายหลักการ FIFO

Queue ใช้หลัก FIFO หรือ First In, First Out หมายถึงคนที่เข้าคิวก่อนจะได้ออกจากคิวก่อน เช่น P001 เข้าก่อน P002 ดังนั้น P001 จึงได้รับบริการก่อน และหลังจาก P001 กับ P002 ออกแล้ว P003 จึงเป็นคนถัดไป